



OOP MASTERDOC

Juhayra Khan

BCS Batch 2025

(remember me in ur duas plss)

Past Paper

LAB Mock Paper Spring 2025

Question 01:

Problem Statement

After becoming a university celebrity, Christie's popularity soared, and she now has a vast Given a system of linear equations:

$$\begin{aligned}a_{11} * x_1 + a_{12} * y_1 + a_{13} * z_1 &= b_1 \\a_{21} * x_2 + a_{22} * y_2 + a_{23} * z_2 &= b_2 \\a_{31} * x_3 + a_{32} * y_3 + a_{33} * z_3 &= b_3\end{aligned}$$

You have to solve them using **Cramer's Rule**.

Steps of Cramer's Rule:

Step 1: Write the coefficient matrix of the system (call this matrix A).

Step 2: Compute the determinant of the coefficient matrix, $|A|$. If $|A|$ is not zero, you may continue; otherwise, Cramer's rule is not applicable here.

Step 3: Suppose the first variable of the system is x . Then write the matrix A_x as follows: substitute the first column of A with the matrix B . Now compute the determinant of A_x , that is $|A_x|$.

Step 4: Now for A_y , substitute the second column of A with the matrix B . Now compute the determinant of A_y , that is $|A_y|$.

Step 5: Finally, for A_z , substitute the third column of A with the matrix B . Now compute the determinant of A_z , that is $|A_z|$.

Step 6: Now you can find the values of your variables as follows:

$$x = \frac{|A_x|}{|A|}, y = \frac{|A_y|}{|A|}, z = \frac{|A_z|}{|A|}$$

Implementation Steps:

Note: You have to solve this question using Dynamic Memory Allocation only.

Step 1: Implement the function `int calculateDeterminant(int matrix[3][3])`, which takes a 3x3 array as input and calculates its determinant.

Step 2: Implement the function `void solveEquations(int A[3][3], int B[3], int X[3])`, which takes a 3x3 matrix A , a 3x1 matrix B , and a 3x1 matrix X . This function calculates the values of the variables using Cramer's rule and stores the result in matrix X such as:

$$\begin{aligned}X[0] &= \text{value of variable } x \\X[1] &= \text{value of variable } y \\X[2] &= \text{value of variable } z\end{aligned}$$

This function should use the above-implemented function `int calculateDeterminant(int array[3][3])` for the calculation of all determinants.

Input:

Co-efficient Matrix A as 3x3 array

A11	A12	A13
A21	A22	A23
A31	A32	A33

Constant Matrix B as 3x1

B1
B2
B3

Output :

Variable Matix X as 3x1

X
Y
z

Question 02:

Problem Statement

Given a positive integer n, print a hexagonal pattern using start with each side containing the sum of previous two numbers starting from 1. You must implement this question using recursion only (any type of loop is not allowed).

Sample Output:

Enter the total Number:4

```

      1
    1 1
  2 2 2
3 3 3 3 3
5 5 5 5 5
8 8 8 8 8
13 13 13 13 13
    21 21
      33 33
        54
          2
```

Question 03:

Problem Statement

Declare a class Block. A block, as you all know, is a cubical container. It has the following attributes:

- Length
- Width
- Height

In addition to these, a block can be made of different materials (e.g., wood, card, metal, etc.).

Furthermore, a block can have different colors. Declare these as member variables of the class.

1. Provide a default constructor and a parameterized constructor for the Block that takes all necessary values as arguments, with the material as optional (if it is omitted, the block is considered to be made of card – default value for the material).
2. Provide getters for all attributes and setters for each, except for the material (the material of the block cannot be changed after it has been created!!!).
3. Provide a function getVolume() that calculates and returns the value of the volume of the block.
4. Also, provide another function getSurfaceArea() that calculates and returns the surface area of the block.
5. Provide a print function that prints the following about the block:
 - Length
 - Width
 - Height
 - Material
 - Color
 - Volume
 - Surface Area
6. Inside main, allocate memory for 5 objects (using an array). Read the necessary values from the user to populate the array.
7. Call the functions Print() and Volume to display the data of the blocks you just saved in the array.
8. Read the index and the new height from the user. Ask the user to provide the index of the block to change its height by creating a function update(Block b[], int size, int index, double height). Update the height of the block present at the index provided by the user, save the updated height right there, and print it again.

Spring 24 Final

Q1: The Galactic Command Protocol

In the futuristic galaxy of FAST (Federation of Allied Star Territories), peace and order are preserved by highly advanced robotic agents known as Galactic Units. These units are deployed to places and p to perform various roles such as exploration, defense, communication, and diplomacy. Each unit has certain capabilities. Some can fly across planetary atmospheres and asteroid fields, others can communicate interstellar signals, and a few are trained for high-stakes combat operations.

To manage these specialized units. The United Planets established a centralized system known as the Galactic Command Center (GCC). This system is designed for reliability and consistency, ensuring that only one instance of the Command Center exists at any given time. The GCC is responsible for deploying units to various missions and ensuring they are equipped for their specific roles.

The units under the GCC's command are categorized based on their capabilities. For example, the SkySec reconnaissance unit specializes in high-speed scouting missions, flying across vast star fields and transmitting crucial intelligence. PeaceDrone, a diplomatic unit, is equipped to send peaceful messages and promote communication between civilizations, while the TitanBot is a combat-ready unit, trained for engaging and defending against hostile forces.

The units deployed by the GCC include reconnaissance units, such as the SkyScout, which combine the ability to fly and communicate. These units are responsible for exploring distant regions of space and relaying information back to the command center. Battle Units, like the TitanBot, are equipped to both fly and engage in combat. These units are prepared to neutralize threats and defend planets from invaders. Finally, Diplomatic units such as the PeaceDrone, focus exclusively on communication. They work to foster peaceful relations between different species and civilizations in the galaxy.

SkyScout is one of the primary reconnaissance units, but there are two distinct versions of it. Each serves a specialized role. SkyScout Alpha is designed for high-speed missions, quickly covering vast distances to sun star systems. In contrast, SkyScout Beta is equipped with advanced sensors to detect anomalies (atmospheric geological) and a high-efficiency communication system to relay information back to the Command Center real-time.

The TitanBot, another key unit in the GCC arsenal, is a specialized combat unit that can both fly and engage in high-intensity battles. The TitanBot is a massive battle unit capable of neutralizing enemy forces and defending the galaxy from invaders. Equipped with advanced targeting systems and powerful weapons, it can fly into combat zones and engage enemies with precision, ensuring the safety of the United Planets' territories.

The PeaceDrone represents the diplomatic arm of the Galactic Units. While it is not designed for combat, the Peace Drone plays a vital role in maintaining harmony across the galaxy. Equipped with long-range communication technology, it can send diplomatic messages, mediate conflicts, and facilitate peaceful interaction between different civilizations. Its primary mission is to foster peace, cooperation, and mutual respect, even in times of tension.

The Galactic Command Center is the heart of the United Planets' defense and exploration system. It guarantees centralized control, ensuring the efficient deployment of reconnaissance units, battle units, and diplomatic units. The Command Center can deploy Recon Units like SkyScout Alpha and SkyScout Beta, which not only fly but also relay critical data about their surroundings. The GCC can also deploy Battle Units, like the TitanBot, neutralize threats and

protect vital assets. Finally, the Command Center can deploy Diplomatic Units, like PeaceDrone, to maintain peace and ensure open communication with potential allies or negotiating parties.

Output:

Galactic Command Center

=== Deploying Recon Unit

SkyScout soaring across star fields with sonic speed!

SkyScout transmitting: Scanning complete

SkyScout soaring across star fields!

Page 2 of 6

Scout transmitting atmospheric & geological data: Scanning complete

Deploying Battle Unit ==

Titan Bot launching with thrusters!

TitanBot engaging enemy unit: Alien Invaders

Deploying Diplomatic Unit ==

PeaceDrone sending diplomatic message: Greetings from the United Planets!

a. For the above case study, generate a class diagram [20 marks]

b. Implement the identified classes including their member functions. [10 Marks]

Q2: Railway Automation Network

The Government of Pakistan has deployed a national system known as RAN (Railway Automation Network) to manage modern railway operations across the country. This AI-powered system automates the control of smart trains, cargo bots, and security patrol units. All units and operations are monitored and managed by a centralized command system called the Central Operations Core (COC). It is designed to be a single, centralized instance, responsible for deploying and managing autonomous rail units across the country's railway routes.

How the System Works:

The RAN system consists of three key types of smart railway units, each with a different responsibility. Each unit operates independently and is deployed by the COC, which keeps track of their missions using a logging system.

Central Operations Core (COC)

This is the brain of the system.

- It receives deployment requests and selects the appropriate unit.
- It logs mission activity using an internal component (Logger).
- It can store and manage multiple units deployed to different cities or tracks.
- Only one instance of the COC should exist during the entire runtime.

ExpressBot (Passenger Service)

- Handles high-speed passenger routes between major cities like Lahore, Islamabad, and Karachi.
- It initiates journeys, auto-stops at smart stations, and reports real-time passenger flow to COC.

CargoBot (Freight Transport)

- Delivers industrial goods to economic hubs like Faisalabad and Gwadar.
- It calculates load capacity, ensures secure delivery, and transmits transport status.

RailGuard (Security Patrol)

- Patrols vulnerable railway sectors and stations.
- Detects threats like track damage or unauthorized personnel.
- Sends safety alerts back to the COC.

Logger

Internally used by the COC to record and track every mission carried out by each bot.

It is not shared with any other class directly - only the COC uses it.

What You Must Do:

You are expected to design and implement this system in C++ using object-oriented programming principles. You must define the appropriate class relationships and responsibilities yourself.

Classes You Must Create:

CentralOperationsCore

- RailUnit
- ExpressBot
- 0124
- CargoBot
- RailGuard
- Logger

Functions You Must Implement:

- execute Task
- monitor PassengerFlow
- calculateCargoLoad
- perform Security Scan
- getUnitInfo
- deployUnit.

You must assign these functions to the appropriate classes and decide:

- Which ones are overridden
- Which ones are virtual or pure virtual
- Which ones are private, public, etc.

Required Operator Overloading:

Implement this overloaded operator in your design: adding a RailUnit to the Central Operations Core using +=

UML Requirement:

You must draw and submit a UML class diagram representing:

- All classes and their names
- Relationships between them (e.g., inheritance, composition)
- Main functions (no code, just signatures)

Important Note on Grading:

Marks will be awarded based on:

- Correct and efficient use of OOP concepts (inheritance, composition, aggregation, polymorphism, operator overloading)
- Logical structure and maintainability of code

Your main function body will somehow look like this.

```
int main() {
    CentralOperations Core coc;
    coc += new ExpressBot ("EXP01", "Lahore to Karachi");
    coc += new CargoBot ("CRG12", "Gwadar to Faisalabad");
    coc += new RailGuard("SEC09", "Sukkur to Multan");
    coc.executeAllUnits();
    return 0;
}
```

The output of the above main function should look like this.

[LOG] Unit deployed: Unit ID: EXP01, Route: Lahore to Karachi

CRG12, Route: Gwadar to Faisalabad [LOG] Unit deployed: Unit ID:

SEC09, Route: Sukkur to Multan [LOG] Unit deployed: Unit ID:

ExpressBot initiating journey: Lahore to Karachi ExpressBot auto-stopping at smart station.

[LOG] Task completed for: Unit ID: EXP01, Route: Lahore to Karachi

CargoBot preparing freight transport on: Gwadar to Faisalabad CargoBot calculating cargo load capacity...

[LOG] Task completed for: Unit ID: CRG12, Route: Gwadar to Faisalabad RailGuard starting patrol on: Sukkur to Multan

RailGuard scanning for anomalies...

[LOG] Task completed for: Unit ID: SEC09, Route: Sukkur to Multan

PF Concepts

Pointers

Helpful Tips:

```
// The refer and derefer operators (* and &) have the absolute lowest precedence compared to everything else;
```

Pointer Arithmetic for Array Traversal

Problem:

Write a function that finds the sum of elements in an integer array using pointer arithmetic. Do not use array indexing. You are only allowed to use pointer dereferencing and pointer arithmetic.

```
int sum(int *arr, int n);
```

Implementing strdup (String Duplicate)

Problem:

Write a function my strdup that duplicates a given string by allocating memory and copying the string into the newly allocated memory. The function should return the pointer to the duplicated string.

```
char * strdup(const char *str);
```

- The function should handle NULL input by returning NULL.

Pointer to Pointer Swap

Problem:

Write a function that swaps the values of two integers using a pointer to pointer approach. You are allowed to pass the addresses of two integer variables using pointer to pointer.

```
void swap(int **x, int **y);
```

Test the function by creating two integer variables and printing their values before and after the swap.

Dynamic Memory Allocation for 2D Array

Problem:

Write a program that dynamically allocates memory for a 2D array of integers. The program should also populate the array with values, and then free the allocated memory.

```
int **create2DArray(int rows, int cols); void fill2DArray(int **arr, int rows, int cols);
```

```
void free2DArray(int **arr, int rows);
```

- create2DArray should dynamically allocate memory for the array. - fill2DArray should fill the array with values (for example, set arr[i][j] = i * j). - free2DArray

should free the allocated memory.

Function Pointer to Implement Basic Calculator

Problem:

Create a simple calculator program that uses function pointers. The program should perform addition, subtraction, multiplication, and division using function pointers.

```
int add(int a, int b) { return a + b; } int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; } int divide(int a, int b) { return a / b; }
}
```

int (*operation[])(int, int) = {add, subtract, multiply, divide}; Test the program by performing each operation on two integers.

Custom calloc Implementation

Problem:

Write your own implementation of the calloc function. The function should allocate memory for an array of elements and initialize all the bytes to zero.

```
void *my calloc(size_t num elements, size_t element size);
```

- The function should return a pointer to the allocated memory.
- If either num elements or element size is 0, return NULL.

1122

Find the Maximum Element in an Array using Pointer Arithmetic

Problem:

Write a function that finds the maximum element in an integer array using pointer arithmetic. Do not use array indexing; only pointer dereferencing and pointer arithmetic should be used.

```
int findMax(int *arr, int n);
```

- The function should return the value of the maximum element in the array.

String Concatenation Using Pointers

Problem:

Write a function that concatenates two strings using pointer manipulation. Do not use any built-in string functions like strcat.

```
void my_strcat(char *dest, const char *src);
```

- The function should append the src string to the dest string. - Make sure to handle null-terminated strings properly.

Detecting Memory Leaks

Problem:

Write a program that simulates a memory leak by allocating memory but failing to free it. Implement a simple function that detects if memory was allocated but not freed (hint: this will be a simplified detection).

```
void memoryLeak(int size) { // Allocate memory and forget to free it } int
```

```
detectLeak() { // Return 1 if leak is detected, otherwise 0 }
```

- The function detectLeak will not actually detect real memory leaks but should simulate checking if memory was allocated without freeing it.

Reversing an Array using Pointer Arithmetic

Problem:

Write a function that reverses an array in place using only pointer arithmetic. Do not use array indices.

```
void reverseArray(int *arr, int n);
```

- The function should reverse the elements of the array arr of size n.

Circular Buffer Implementation Using Pointers

Implement a circular buffer using pointers. The buffer should support the following operations: 1. Adding an element to the buffer (enqueue). 2. Removing an element from the buffer (dequeue). 3. Checking if the buffer is full or empty.

Use the following functions:

```
void initBuffer(CircularBuffer *cb, int size); void enqueue(CircularBuffer *cb, int value);
```

```
int dequeue(CircularBuffer *cb); int isFull(CircularBuffer *cb); int isEmpty(CircularBuffer *cb);
```

Use dynamic memory allocation to manage the buffer.

Implementing Linked List Traversal Using Pointers

Problem:

Write a program to implement traversal of a singly linked list using pointers. Each node should store an integer value.

```
typedef struct Node { int data; struct Node *next; } Node; Functions to implement:  
Node *createNode(int value); void traverseList(Node *head); void freeList(Node  
*head);
```

Ensure proper memory management by freeing all nodes at the end of the program. 1125

Pointer-Based Matrix Multiplication

Problem:

Write a program to multiply two matrices using pointers. Use dynamic memory allocation for the matrices.

```
void matrixMultiply(int **mat1, int rows1, int cols1, int **mat2, int rows2, int cols  
- Validate that the number of columns in mat1 matches the number of rows in mat2. -  
Allocate memory for the result matrix dynamically.
```

Pointer-Based String Reverse

Problem:

Write a function that reverses a string in place using pointer manipulation

```
void reverseString(char *str);
```

- The function should handle null-terminated strings and ensure proper memory safety.

Multi-Level Pointer Manipulation

Problem:

Write a program to demonstrate multi-level pointers by dynamically creating a variable and modifying its value using a triple pointer.

```
void manipulateTriplePointer(int ***ptr);
```

- Use this program to show the value at the original pointer being changed indirectly.

Dry Run 1.

```
int arr[] = {5, 10, 15}; int* p = arr;  
// ++*p+=3;  
  
// *++p += 3;  
  
// comment out either one of these p++;  
cout << arr[0] << " " << arr[1] << " " << arr[2];
```

Dry Run 2.

```
#include <iostream> using namespace std;

int operate (int a, int b){ return (a * b);
}
float operate (float a, float b){ return (a / b);
}

int main (){
int x = 6, y = 7; float s = 4.0, u = 2.0;

cout << operate (x, y) <<endl<< operate (s, u); return 0;
}
```

Dry Run 3.

```
#include <iostream> using namespace std; int main() {
int arr[2] [3]={
{1,2,3},
{4,5,6}
};
int (*ptr) [3]=arr;
// cout<<ptr[1] [2];
return 0;
}
```

Dry Run 4.

```
#include <iostream> using namespace std;
int* findNull(int* array){
while(*(array++)!='\0'){ } return array--;
}

int main() {
int arr[10]={1, 2, 3, 4, 5, '\0', 1};
int *nullPtr = findNull(arr); while(nullPtr>arr){
cout << *(--nullPtr) << endl;
}
return 0;
}
```

```
}
```

Dry Run 5.

```
char s[8]={'C','+', '\\n','+', '\\r','!', '\\0'};  
cout<<s;
```

Dry Run 6.

```
int arr[] = {2, 4, 6, 8}; int *p = arr;  
cout << *p++ << " " << *p++ << " " << *p;
```

Dry Run 7.

```
const int s=3;  
int* listMystery(int list[][::s]){ int i = 1, k=0;  
int *n = new int[::s]; for(int i=0;i<::s;++i)  
n[i]=0;  
while(i < ::s){  
int j = ::s - 1;  
while(j >= i){  
n[k++]=list[j][i] * list[i][j];  
j = j - 1;  
}  
i = i + 1;  
}  
return n;  
}  
void displayMystery(int * arr){  
cout<<"[ ";  
for(int i=0;i<::s;++i)  
cout<<arr[i]<<(i!= (::s - 1)? " , " : " ");  
cout<<"]"<<endl;  
}  
int main(){  
int L[][::s] = {{8, 9, 4}, {2, 3, 4}, {7, 6, 1}};  
int *ptr=listMystery(L);
```

```
displayMystery(ptr);
delete [] ptr;
return 0;
}
```

Dry Run 8.

```
#include <iostream> using namespace std;
int findNull(int* array){ while(*(array++)!='\0');
return array--;
}
int main() {
int arr[10] = {1, 2, 3, 4, 5, '\0', 1};
int *nullptr = findNull(arr); while(nullptr>arr)
cout << *(--nullptr) << endl;
return 0;
}
```

Dry Run 9.

```
#include <iostream> using namespace std; int main()
const int x = 5; int y=10;
const int ptr=&y; cout <<<< ptr<<endl;
*ptr++; cout << ptr; ptr=&x; cout << ptr;
return 0;
}
```

Dry Run 10.

```
void allocateMemory(int** ptr) {
*ptr = new int;
++ptr = 42;
}
int main() {
int* value = nullptr; allocateMemory(&value);
cout <<< &value<<" "<<value<<" "<<*value<< endl; delete[] value;
value= nullptr; return 0;
}
```

General Class Questions

Implement 1

Implement the Polynomial class for the given main function, including setters and getters for every attribute you create, you are not required to write any outputs.

```
#include <iostream>
using namespace std;
class Polynomial {
    // Your code here
};

int main() {
    Polynomial p(3); // Polynomial up to x^3
    p.setX(0, 5), p.setX(1, 3), p.setX(2, 9), p.setX(3, -3);
    p.display(); // Expected: -3x^3 + 9x^2 + 3x + 5
    Polynomial q(2); // Polynomial up to x^2
    q.setX(0, -5), q.setX(1, 2), q.setX(2, -3);
    q.display(); // Expected: -3x^3 + 6x^2 + 5x + 0
    q.add(p);
    q.display(); // Expected: -3x^3 + 6x^2 + 5x + 0
    p.display(); // Expected: -3x^3 + 9x^2 + 3x + 5
    p.freeMemory(); // Frees Heap Memory
    q.freeMemory(); // Frees Heap Memory
    return 0;
}
```

Implement 2

Implement the Vector class for the given main function, including setters and getters for every attribute you create, you are not required to write any outputs.

Note : To take dot product, multiply corresponding elements of the vectors(e.g x1 with y1, x2 with y2 etc. and add the results)

```
#include <iostream>
using namespace std;

class Vector {
    // your code here
};

int main() {
    Vector p(4); // Vector in R4
    p.setX(0, 5), p.setX(1, 3), p.setX(2, 9), p.setX(3, -3);
    p.display(); // Expected: (5, 3, 9, -3)
    p.setX(9, -3); // invalid index
    Vector q(3); // Vector in R3
    q.setX(0, -5), q.setX(1, 2), q.setX(2, -3);

    q.display(); // Expected: (-5, 2, -3)
    cout << q.dot(p) << endl; // Expected: -46
    q.display(); // Expected: (-25, 6, -27 0)
}
```



```
p.freeMemory(); // Frees Heap Memory  
q.freeMemory(); // Frees Heap Memory  
return 0;  
}
```

Constructors & Destructors

default, parameterized, copy

Quick Notes:

- Initialiser List runs in the order that items are listed in the class definition.

Example:

```
class A{
    int* arr;
    int i;
public:
    A(int n): i(n), arr(new int[i]) {}
};
```

Here, arr will initialise first with i at the default 0.

Dry Run 1

```
#include<iostream>
using namespace std;
class D{
public:
    D(){ cout<<"D C"<<endl;}
    ~D(){ cout<<"D D"<<endl;}
};
class E{
    D &d;
public:
    E(D &d):d(d){ cout<<"E C"<<endl;}
    ~E(){ cout<<"E D"<<endl;}
};
class F{
    E * e;
    D * d;
public:
    F(E* e,D * d):e(e),d(d){cout<<"F C"<<endl;}
    F(D * d):d(d),e(new E(*d)){ cout<<"F C"<<endl;}
    ~F(){
        cout<<"F D"<<endl;
    }
};

int main() {
    E e(d);
    {
        F f(&d);
    }
    F f(&e,&d);
    cout<<"-----"<<endl;
```

```
return 0;
}
```

Dry Run 2

```
#include<iostream>
using namespace std;
class A{
public:
A(){ cout<<"A C ";}
~A(){ cout<<"A D"<<endl;}
};
class B{
A a1;
public:
B(){ cout<<"B C ";}
~B(){ cout<<"B D"<<endl;}
};
class C{
A * a;
B b;
public:
C(B &b):b(b),a(new A){ cout<<"C C ";}
~C(){
delete a;
cout<<"C D"<<endl;
}
};

int main() {
B b;
C c(b);
A a;
return 0;
}
```

Functions

Dry Run 1

```
#pragma once
#include <iostream>
using namespace std;

class Number {
    int* x;
public:
    Number(int a) {
        cout << "Hello Number" << endl;
        if (!a) {
            x = new int(a);
        }
        else if (a % 2 != 0) {
            *this = Number(--a);
        }
        else {
            x = new int(a);
            *this = Number(*this);
        }
    }
    Number(const Number& n2) {
        cout << "Copy Number ";
        x = new int(*(n2.x));
    }
}
```

```

        cout << *x << endl;
    }
    Number& operator=(Number n2) {
        x = new int(*(n2.x));
        return *this;
    }
    int operator--() {
        return (*x)++;
    }
    int& operator*() {
        return *x;
    }
    void operator delete(void* ptr) {
        cout << "Deleting X" << endl;
    }
    ~Number() {
        cout << "Bye " << *x << endl;
    }
};

int main() {
    Number n1(5);
    Number* n2 = new Number(--n1 + 1);
    Number n3 = *n2;
    cout << --(*n2) << endl;
    cout << --n1 << endl;
    delete n2;
    n2 = new Number(n3);
    return 0;
}

```

Dry Run 2

```

#pragma once
#include <string>
#include <iostream>
using namespace std;

class ShoppingCart {
private:
    static int itemsCount;
    string itemName;
public:
    ShoppingCart(string n = "1") :
        itemName(n) {
        cout << " Item= " << itemName
            << " " << ++itemsCount << endl;
    }
    void setItemsCount(int count) {
        itemsCount = count;
    }
    ShoppingCart* operator&() {
        cout << " Item= " << itemName
            << " " << ++itemsCount << endl;
        return this;
    }
    static int getItemsCount() {
        return itemsCount;
    }
    ~ShoppingCart() {
        itemsCount--;
    }
}Sc_one;

int ShoppingCart::itemsCount = 0;

int main() {
    ShoppingCart Sc_two("2");
    Sc_one.setItemsCount(5);
    ShoppingCart* scfour = &Sc_one;
    {
        ShoppingCart sc_Three("3");
        scfour = new ShoppingCart("6");
        scfour = &sc_Three;
    }
    Sc_two.setItemsCount(10);
    cout << ShoppingCart::getItemsCount();
    return 0;
}

```

Dry Run 3

```
#pragma once
#include <iostream>
using namespace std;

class ShoppingCart {
private:
    static int itemsCount;
    string itemName;
public:
    ShoppingCart(string n = "1") :
        itemName(n) {
        cout << " Item= " << itemName
            << " " << ++itemsCount << endl;
    }
    void setItemsCount(int count) {
        itemsCount = count;
    }
    ShoppingCart* operator&() {
        cout << " Item= " << itemName
            << " " << ++itemsCount << endl;
        return this;
    }
    static int getItemsCount() {
        return itemsCount;
    }
    ~ShoppingCart() {
        itemsCount--;
    }
}Sc_one;

int ShoppingCart::itemsCount = 0;

class Matrice {
public:
    Matrice(int r, int c) :
        rows(r), columns(c), ptr(new int[r * c]) {
    }
    ~Matrice() {
        cout << *this;
        delete[] ptr;
    }
    void operator=(int* lmat) {
        for (int i = rows * columns - 1, j = 0; i >= 0; --i, j++)
            ptr[j] = lmat[i];
    }
    int& operator()(int i, int j) {
        return ptr[j + i * columns];
    }
    int& operator[](int i) {
        return ptr[i];
    }
    friend ostream& operator <<(ostream& cout, Matrice& m);
private:
    int rows, columns;
    int* ptr;
};

ostream& operator <<(ostream& cout, Matrice& m) {
    cout << "Matrix = " << endl;
    for (int i = 0; i < m.rows * m.columns; ++i) {
        cout << m[i] << " ";
        if ((i + 1) % m.columns == 0)
            cout << endl;
    }
    return cout;
}

int main() {
    int a[] = { 10, 20, 30, 40, 50, 60, 70, 80 };
    Matrice v(4, 2); v = a; int z = ++v(2, 1);
    cout << " Z= " << z << endl;
    return 0;
}
```

UML

Case Studies

The Galactic Command Protueui

In the futuristic galaxy of FAST (Federation of Allied Star Territories), peace and order are preserved by highly advanced robotic agents known as Galactic Units. These units are deployed to planets and planets to perform various roles such as exploration, defense, communication, and diplomacy. Each unit possesses certain capabilities. Some can fly across planetary atmospheres and asteroid fields, others can communicate interstellar signals, and a few are trained for high-stakes combat operations.

To manage these specialized units, the United Planets established a centralized system known as the Galactic Command Center (GCC). This system is designed for reliability and consistency, ensuring that only one instance of the Command Center exists at any given time. The GCC is responsible for deploying units to various missions and ensuring they are equipped for their specific roles.

The units under the GCC's command are categorized based on their capabilities. For example, the SkyScout reconnaissance unit specializes in high-speed scouting missions, flying across vast star fields and transmitting crucial intelligence. PeaceDrone, a diplomatic unit, is equipped to send peaceful messages and maintain communication between civilizations, while the TitanBot is a combat-ready unit, trained for engaging and defending against hostile forces.

The units deployed by the GCC include reconnaissance units, such as the SkyScout, which combine the ability to fly and communicate. These units are responsible for exploring distant regions of space and relaying critical information back to the command center. Battle Units, like the TitanBot, are equipped to both fly and engage in combat. These units are prepared to neutralize threats and defend planets from invaders. Finally, Diplomatic units such as the PeaceDrone, focus exclusively on communication. They work to foster peaceful relations between different species and civilizations in the galaxy.

SkyScout is one of the primary reconnaissance units, but there are two distinct versions of it, each serving a specialized role. SkyScout Alpha is designed for high-speed missions, quickly covering vast distances to various star systems. In contrast, SkyScout Beta is equipped with advanced sensors to detect anomalies (atmospheric geological) and a high-efficiency communication system to relay information back to the Command Center in real-time.

The TitanBot, another key unit in the GCC arsenal, is a specialized combat unit that can both fly and engage in high-intensity battles. The TitanBot is a massive battle unit capable of neutralizing enemy forces and defending the galaxy from invaders. Equipped with advanced

targeting systems and powerful weapons, it can fly into com zones and engage enemies with precision, ensuring the safety of the United Planets' territories.

The PeaceDrone represents the diplomatic arm of the Galactic Units. While it is not designed for combat, the Peace Drone plays a vital role in maintaining harmony across the galaxy. Equipped with long-range communication technology, it can send diplomatic messages, mediate conflicts, and facilitate peaceful interaction between different civilizations. Its primary mission is to foster peace, cooperation, and mutual respect, even times of tension.

The GalacticCommand Center is the heart of the United Planets' defense and exploration system. It guarantees centralized control, ensuring the efficient deployment of reconnaissance units, battle units, and diplomatic units. The Command Center can deploy Recon Units like SkyScout Alpha and SkyScout Betaz, which not only fly but also relay critical data about their surroundings. The GCC can also deploy Battle Units, like the TitanBot, to neutralize threats and protect vital assets. Finally, the Command Center can deploy Diplomatic Units, like PeaceDrone, to maintain peace and ensure open communication with potential allies or negotiating parties.

Food Delivery System

Case Study: Online Food Delivery System Problem Statement:

The case study titled "Online Food Delivery System" is a platform for the purpose of ordering food online from a local restaurant or food cooperative of our choice through a Website or App.

This application provides us with complete information regarding any restaurant or food cooperatives around a certain radius of users' locations, that have signed up as a grocer in the platform.

It features a familiar and well-thought-out and attractive user interface, much like ordering consumer goods online, users can have personalized accounts in order to make the frequent ordering of food made convenient. A user can search for a favourite restaurant, usually filtered via type of cuisine and choose from available items, and choose delivery or pick-up.

In this business, the online food delivery platform acts as the intermediate between restaurants or hotels and end-user or customer. And it charges a commission to the restaurants on the basis of orders.

While users pay a delivery fee, the platform earns through restaurants that pay a commission for each delivery, which is then split among the delivery partner and the company. The payment gateway of the platform accepts credit/debit cards, UPI or Cash on delivery.

Furthermore, users can pre-order based on their plan for their meal and are organized. The system can also provide a Scan & Order feature from their favourite restaurants. The major advantage of this proposed system is to have the Take Away Ordering facility, a win-win solution for restaurant owners and customers. The following are the brief description of the functions achieved through this case study with the roles of end-users:

.....

End-Users:

User/Customer: Need food for appetite and also place various issues/requests to the employees of the platform.

Grocer/Seller: To provide the food to the users who placed the order, through the delivery man of the platform.

Delivery man: To transport the food to the preferred location of the users.

The One Who Logs Everything

A military intelligence division operates a classified logging bureau. This bureau is special: there must be exactly one instance of it running at any given time. If two colonels accidentally spin up two loggers simultaneously, mission records collide, timestamps corrupt, and – as one general once put it – 'we end up with two different histories of the same war.'

The bureau records entries at different severity levels. An entry is the most basic unit – it has a timestamp, a source identifier, and a message. A classified entry inherits from entry but also carries a clearance level and an encryption flag. A field-report entry inherits from classified entry and adds a terrain tag, a mission code, and a list of casualty counts. Three levels of inheritance. Each level adds specificity without breaking what came before.

The bureau itself – the Logger – must be accessed globally by every system in the game: the enemy spawner, the scoring engine, the boss fight controller. None of them should be allowed to create their own Logger. They all go through one door.

There is one final constraint: the Logger stores all entries in a fixed-size circular buffer of 256 slots. When it fills up, new entries overwrite the oldest ones silently. This means the buffer must also be a single shared resource – it lives inside the Logger singleton and nowhere else.

The Soldier Who Was Too Many Things

An elite military academy produces three types of operatives: Combat Specialists, Tech Specialists, and Support Specialists. All three ultimately trace their training back to the same foundational Academy Trainee class – a base that gives every operative their ID, their rank, their health rating, and their deployability flag.

A new directive creates a fourth classification: the Hybrid Operative. This operative completed both the Combat and Tech training tracks in full. The institution didn't foresee the problem: if Combat Specialist inherits from Academy Trainee, and Tech Specialist also inherits from Academy Trainee, then the Hybrid Operative – inheriting from both – now contains two completely separate copies of Academy Trainee inside it. Two IDs. Two health ratings. Two deployability flags. The drill sergeant calls this 'a soldier who doesn't know which version of himself to salute with.'

The fix is virtual inheritance. Both Combat Specialist and Tech Specialist must declare their inheritance from Academy Trainee as virtual, so that when Hybrid Operative inherits from both, the compiler merges them into a single shared sub-object. Now there is one ID. One health rating. One soldier.

The Hybrid Operative must also correctly handle the ambiguity in `performDuty()` – both parent classes define it, and the Hybrid must resolve which version to call, or provide its own that calls both explicitly in a defined order.

The Difference Between Owning and Borrowing

A field squadron is assembled before every mission. The squadron carries its own radio – a hardwired device bolted into its command structure. If the squadron is disbanded, the radio goes with it. No radio survives a disbanded squadron. The two are inseparable. This is ownership.

The same squadron may be assigned a sniper. The sniper exists independently – she has a name, a record, a home base. She can be reassigned to a different squadron tomorrow, or pulled off mission entirely. If the squadron is disbanded she walks away. The squadron referenced her; it never owned her.

In the armoury, a weapon crate is assembled for dispatch. It contains exactly three items: a firearm, an explosive, and a melee tool. These items were manufactured specifically for this crate – they carry the crate's serial number etched into the handle. When the crate is destroyed in the field, all three items are destroyed with it. Composition.

The armoury also keeps a roster of available soldiers. The roster holds pointers to soldiers who exist independently. Soldiers rotate. The roster merely references them. If the roster is deleted, the soldiers don't vanish – they report to HQ. Aggregation.

Your task is to model both relationships in C++ for a military resource management subsystem, distinguish them in code through pointer ownership semantics, and demonstrate the behavioural difference through destructors.

The Factory That Builds Enemies

The war machine generates enemies. Not randomly – intelligently. A central factory receives a terrain code and a difficulty tier, and it manufactures the appropriate threat. The factory does not know the exact type at compile time. It only knows the abstract interface. At runtime, it produces a Ground Threat, an Aerial Threat, or an Aquatic Threat – each with its own movement pattern, its own attack pattern, and its own loot table.

Each threat, regardless of its type, exposes exactly the same interface to the game engine: `spawn()`, `update()`, `attackPlayer()`, and `dropLoot()`. The engine calls these blindly through a base pointer. It doesn't know whether it's a tank, a helicopter, or a submarine. It only knows: this is a Threat, and threats do these things.

The factory itself is abstract. Three concrete factories exist: `GroundThreatFactory`, `AerialThreatFactory`, and `AquaticThreatFactory`. Each factory implements a single method: `createThreat(int difficulty)` which returns a `Threat*` to the appropriate subclass. This is the Factory Method pattern layered on top of an abstract class hierarchy.

There is one further constraint: the difficulty tier modifies a stat multiplier on every threat. Difficulty 1 is the default. Difficulty 2 doubles health. Difficulty 3 doubles health and fire rate. This multiplier must be applied polymorphically inside each threat's constructor via a virtual `applyDifficulty()` method called from the base constructor.

The Mission Control System

A government defence agency runs an automated Mission Control System. This system manages everything: it tracks active operatives, assigns missions, records outcomes, and ensures that at all times there is exactly one central command node – one `MissionControl` object that every subsystem consults.

Each operative belongs to one of three branches: infantry, reconnaissance, or demolitions. All three branches share a common base: the `Operative` class which holds an ID, a callsign, a health status, and an active/inactive toggle. Infantry adds firepower and unit count. Reconnaissance adds stealth mode and scan radius. Demolitions adds charge count and blast proficiency. This is a three-branch single-inheritance tree rooted at `Operative`.

A mission is composed of exactly three things: a primary operative (who leads), a weapon loadout (three `WeaponSlot` objects, each owning a `Weapon`), and a report log. The weapon loadout is owned by the mission – when the mission ends, the loadout is destroyed. The primary operative is NOT owned by the mission – she continues to exist independently and may be reassigned.

The `MissionControl` singleton maintains a dynamic list of all missions ever launched. It provides a method to query any mission by ID, and a method to `printSummary()` that polymorphically calls `report()` on every operative's branch-specific version. It also maintains a count of total missions launched since the programme started – a static counter shared across all instances (which is one, because it's a singleton).